⚓ SOVEREIGN VESSEL ECOSYSTEM

Technical Architecture Brief · March 2026

The DBMS Is the Problem. The Vessel Is the Solution.

The traditional stack separates data from logic. Every web application you have ever built contains the same wastes: an ORM to bridge the app to the database, an API layer to bridge the database to the UI, an authentication service to manage who can touch the data, a cloud provider to host all of it, and a monthly invoice for the privilege. The Sovereign Vessel collapses this stack to zero.

A Vessel is a single HTML file. It contains the UI, the logic engine, the storage controller, the ML engine, and the data — all fused. The browser is the only runtime required. There is no backend.

Architecture

Four vessels constitute the current constellation:

Vessel	Size	Dependencies	Role
BiBBOS v6.9	825KB	Zero	Business OS — accounting, CRM, payroll, ML, formation
Sovereign PM v2.4	1,210KB	Zero	Project vessel — lifecycle, charter, budget, key commissioning
The Masthead v5	56KB	Zero	Formation oversight — witnesses, directives, cross-device
server.js	72KB	ws, express, node	Formation hub — stateful registry, backpressure, rate limiting

Formation Hub — WebSocket Architecture

The formation hub runs on server.js at ws://[LAN-IP]:8080/formation. Key design decisions:

- Compact registry: makeRegistryRecord() stores only named fields — entityName, icon, status, healthScore, vesselKey, and a bounded _intentions summary (up to 50, key fields only). Full payloads are never stored in the registry — they are relayed live and eligible for GC immediately.
- Backpressure: safeSend() checks ws.bufferedAmount before every write. Buffer > 512KB triggers a strike. Two strikes terminate the slow client. Prevents the crash scenario where a frozen browser tab fills the Node heap.

- Rate limiting: 80 outbound messages/second per client. 30 non-heartbeat inbound messages/10 seconds per sender. Heartbeats and pings are exempt.
- Departure grace: named vessels get 8 seconds after disconnect before departure is broadcast. Covers mobile screen-lock reconnect. Timer is deduplicated — only one per vesselId at any time.
- Two-tier keepalive: named vessels get 2 missed ping cycles (60s). Anonymous connections get 1 (30s).

FormationTransport v2.4

The transport shim is served from the hub at /formation-transport.js. Key behaviours:

- Single send path: WS only when connected, BC (BroadcastChannel) only when not. Never both. The server fans out cross-device AND same-browser when WS is active.
- Receive path: both BC and WS always active. Dedup window of 500ms suppresses duplicates.
- Reconnect backoff: 2s · 3s · 4.5s · ... · 30s cap. Maximum 20 reconnect attempts.
- Offline queue: up to 64 messages queued while disconnected, flushed as sync burst on reconnect with _offline:true flag.

Sovereign ML Engine

Three genuine ML algorithms running entirely within the vessel — no CDN, no npm, no WebGL:

- Linear regression: OLS on daily revenue time-series. Returns slope, intercept, R^2, and a predict(x) function. Parameters learned from vessel data, not hardcoded.
- Z-score anomaly detection: learns μ and σ from invoice amounts. Flags $|z| > 2.5$ as outliers. Threshold adapts to each vessel's specific billing pattern.
- K-means++ customer segmentation: clusters by purchase frequency, total spend, and days-since-last-order. K-means++ initialisation for stable convergence. k=3 by default, scales to min(3, custData.length).

vesselGenesis — Provenance DNA

Every vessel commissioned since March 2026 carries a sovereignStandard block in its vesselGenesis:

- FAIR fields: findable (vesselId + fingerprint), accessible (zero infrastructure), interoperable (JSON-native), reusable (Sovereign License).
- CARE fields: collectiveBenefit: true, authorityToControl: 'StewardOnly', responsibility: 'EmbeddedInDNA', ethics: 'SovereignByDesign'.
- aiPolicy: allowLocalProcessing: true, allowExternalEgress: false, attributionRequired: true, sovereignBoundary: 'StrictLocal'.

- attribution: academic lineage (DOI: 10.5281/zenodo.3663553), AI Federation, human steward, build period.

Active State Stewardship — Paradigm

Traditional DBMS architecture is passive warehousing: data is stored at rest, retrieved on demand, separated from the logic that uses it. The Sovereign Vessel is Active State Stewardship: data and logic are fused in a single kinetic entity. The vessel does not connect to a database — it is the database. Schema evolution is adding a key. Migrations are obsolete. The vessel adapts to the state it finds within itself.

Sovereignty Score: 96/100

Remaining 4 points: ML Local 10/15 unrealised. aiPolicy.allowLocalProcessing: true is already set. Path: WebLLM / Transformers.js loaded locally (model cached in browser, no CDN call after first load), autonomous bookkeeping classification, Monte Carlo simulation.

Health Monitoring

GET http://[LAN-IP]:8080/api/formation/health returns: uptime, client count, registry size, msgs_total, heap_used_mb, heap_total_mb, rss_mb. A 5-minute console log fires when clients > 0: [FormationHub] © N clients · M registry · heap XMB · rss YMB.

Field-confirmed: heap 9–11MB stable across 16+ simultaneous clients. rss shrinks between GC cycles (healthy). No growth observed after memory leak fixes.

⚓ One Truth. One Vessel. One Pattern.